

Analysis: Hidden Pancakes

Test Set 1

For Test Set 1, the number of pancakes is small enough to do something exponential. One possible way would be to go through every possible order of pancakes. That would be too slow, but we can identify some repetition: any hidden pancakes at any point could be hidden in multiple ways, and that does not affect how we continue the process. Removing the cost of calculating these overlapping cases leads to a [dynamic programming](#) solution.

We can formalize the idea above as follows: instead of remembering exactly the current stack of pancakes as the current state, we can store one of 3 states for each pancake: unused, visible, or hidden. The visible pancakes are always stacked in decreasing order of radius, and the position in the stack of hidden pancakes, as argued above, does not affect the process. Thus, we can define a function f recursively that takes a state description and returns the number of valid ways to finish a pancake stack from that state. Applying f to the state in which all pancakes are visible gives the answer of the problem.

Let's define $f(s)$ recursively, where s is a state. If no pancakes are unused in s , then the number of ways to finish it is simply 1. This is our base case. If there are unused pancakes, we can go through all of them to choose which is the next pancake that should be cooked. If s_i is the state that results from cooking the pancake of radius i centimeters when the state was s , then $f(s) = \sum_i f(s_i)$ where the summation is over pancakes that are unused in s .

Since each pancake can independently be in one of 3 states, the domain of f has 3^N overall states. The non-recursive cost of computing f is a low-degree polynomial on N . The exact degree depends on the implementation, but it is not too hard to do it in linear time, yielding an overall algorithm that runs in $O(3^N \cdot N)$.

We can also use backtracking to solve the problem. We start building the permutations one element at a time, checking that the V_i s for this partial permutation match the ones needed for this test case. It can be proven that the complexity of such a solution is bounded by $O(2^N N^3)$, which is significantly better than $O(N!)$ of the simplest brute force algorithm, and is enough to solve this test set.

Test Set 2

From the input, we can keep track of the current list of visible pancakes. To make things easier, when a pancake gets covered up in the stack, we will view this as the pancake being "removed" from the list of visible pancakes. We can create a list of inequalities relating the sizes of the pancakes. Let P_i be the radius of the i -th pancake added.

First, it's worth noting that each time we add a pancake to the stack, the size of the list of visible pancakes will either increase by 1, stay the same, or decrease. If the size of the list ever increases by more than one, then this case is impossible so our answer is 0.

Let's consider the possible scenarios when we are adding the i -th. When the size of the list increases by 1 ($V_i = V_{i-1} + 1$), we know that the new pancake is smaller than all of the pancakes in our list. We can add the inequality $P_x > P_i$ where x is the pancake that was previously at the end of the list.

If the size of the list stays the same or decreases ($V_i \leq V_{i-1}$), then we know that the new pancake is larger than the last $V_i - V_{i-1} + 1$ pancakes on the list and each of these pancakes gets removed from our list. We can add the inequality $P_i > P_x$ where x is the last pancake we removed from the list. Also, if $V_i > 1$, then the new pancake is strictly smaller than the rest of the pancakes in the list. In this case, we can add the inequality $P_y > P_i$ where y is the first pancake we did not "remove".

Note that in the latter case, we would already have added an inequality saying that $P_y > P_x$. But, this inequality is redundant now that we have the extra inequalities $P_y > P_i$ and $P_i > P_x$. Therefore, we can remove the inequality $P_y > P_x$ to prevent us from having any redundant inequalities that can cause us issues later.

After removing the redundant edges, each pancake is on the right side of at most one inequality in the form of $A > B$. Because of this, the inequalities can be modeled as a tree where we have an edge from A to B if $A > B$. We can then solve for the number of valid cooking orders of N pancakes recursively starting at the root of the tree which is the largest pancake. Note: the largest pancake is whichever pancake was at the beginning of our list at the end.

Let $s(i)$ be the number of pancakes in the subtree rooted at i . Also, let $f(i)$ be equal to the number of valid permutations of the pancake sizes (ranging from 1 to $s(i)$) in the subtree starting at i .

To solve for $f(i)$ we need to count the number of ways we can assign pancake sizes to the subtrees belonging to our direct children. Then, each child subtree can permute itself in $f(j)$ ways (where j is a direct child of i). So, $f(i)$ is the product of all $f(j)$'s and the number of ways we can assign pancake sizes to our subtrees.

Since i is the largest pancake in our subtree, it must be given the largest size. The other $s(i) - 1$ pancake sizes for our subtree can be assigned to any subtree. The number of ways to do this can be counted using [Multinomial Coefficients](#).

If we precompute the factorials and inverse factorials (to allow for division under mod), we can count the number of valid cooking orders in $O(N)$. Building the tree of inequalities also can be done in linear time. This gives us a final time complexity of $O(N)$ (assuming we precompute factorials and inverse factorials in linear time).

An alternate solution

There is a different solution to the problem that requires two observations: (1) the largest pancake (the one with a radius of N cm) can only be placed at position k , where k is the largest integer such that $V_k = 1$ and (2) since the pancake with a radius of N covers all other pancakes, the order of the pancakes before the largest pancake does not impact the V_i 's for the pancakes after the largest pancake. This allows us to split the problem into two independent parts. We solve the left part and the right part separately. For the right part, the largest pancake will always be visible, so we subtract 1 from all V_i in the right part to account for it (note we do not actually do the subtraction, because that would be too slow, but we just implicitly do it). We must also choose which pancakes were in the left and right parts (there are $\binom{N-1}{k-1}$ ways of doing this where the largest pancake was at index k). As base cases: If the range is empty, then there is 1 valid ordering, and if there is no $V_i = 1$, then there are 0 valid orderings. Otherwise, the product of the left part's answer, the right part's answer, and the binomial coefficient is the total number of orderings.

Analysis: Matrygons

Test Set 1

The statement of this problem requires calculating a function from integers to integers. After we select a polygon to use, the remainder of the work is similar to the overall: select more polygons. This almost screams: write a recursive solution!

One way of doing that is with a top-down approach: start by picking the largest polygon, and then pick the rest. For all polygons except the largest, there is the additional restriction of "fitting" into the previous polygon. That is, the number of sides of the new polygon must be a proper divisor of the number of sides of the last one.

We can code that into a simple function that takes a number of sides left t and the size of the last polygon used p : $f(t, p) = 1 + \max_{q:q|p} f(t - q, q)$ (one plus the maximum $f(t - q, q)$ over all q s that are divisors of p) adding $f(0, p) = 0$ as base case, and appropriate conditions that q is indeed a polygon. The answer is then $\max_p f(\mathbf{N}, p)$. This recursion is fast enough for the small bounds of Test Set 1 even if we iterate all possible $3 \leq q \leq \min(p - 1, t)$. Of course, we can avoid that and find the divisors faster by iterating only up to $\sqrt{p}$ and checking both q and p/q , but the optimization is not necessary for Test Set 1, and not sufficient for Test Set 2.

Test Set 2

While [memoization](#) is the first instinct to speed up a recursive function, the function described has too large of a domain for that. A bottom-up approach, on the other hand, is much more suitable.

Consider starting from the smallest polygon. This can be done by using a function that is described by a similar expression, but swapping which of p and q is a parameter and which is iterated over: $f(t, q) = 1 + \max_{p:q|p} f(t - p, p)$. This switch allows for a significant speed up: when checking all possible p we can simply jump by q , dividing the total size of the iteration by q . This is enough to pass Test Set 2 if implemented carefully, but it may be hard to be confident that it is.

We can do better and improve our confidence with the following observation: once we pick a polygon of size q , all future polygons will have sizes multiples of q . Therefore, we can divide the whole problem by q (and allow pseudo-polygons of size 2) by setting $f(t, q) = f(t/q, 1)$. If we are careful about not allowing size 2 for the very first polygon, this leaves us having to calculate only $g(t) = f(t, 1)$, which has a domain of memoizable size. If we add memoization, we have a faster solution, and more importantly, one for which we can pre-compute the entire recursive function and be sure about the running time irrespective of the input.

Analysis: Minimum Sort

In this problem, we are given only one tool to get information about our input: the minimum query. Moreover, the query becomes cheaper when done over long distances. This means a sorting algorithm that normally spends running time finding minimums would align well with our needs. One well known such algorithm is [Selection sort](#).

In Selection sort, we search for the minimum of the full list, put that in its correct position (the beginning), and continue solving recursively searching for the minimum of a suffix of the list, and moving that minimum to the beginning. This can be implemented with $N - 1$ pairs of the minimum query and (possibly) a swap operation as follows:

```
for i := 1 to N-1
  j = minimum_index_of(i, i+1, ..., N)
  if i != j:
    swap(i, j)
```

The minimum queries above are over ranges of sizes $N, N - 1, \dots, 2$. This means the total cost is exactly

$$\sum_{i=2}^N \left\lceil \frac{10^8}{i} \right\rceil.$$

For $N = 100$, this is 418737795, which is less than the limit of 6×10^8 .

Analysis: Retiling

The first simple observation we need to solve this problem is that we never swap two tiles that are showing the same color.

The main not-that-simple observation is that any given tile should either be flipped or swapped (or neither), but never both, regardless of costs. This can be proved by considering all operations that affect tile a : if there is a swap of a and b followed by a flip of a or vice versa, the same effect can be achieved for less cost by simply flipping b (which may have moved by its own swaps in between).

A small addition to the above is that we never want to flip the same tile more than once. Since each tile is only affected by one type of operation, we can assume that all swaps happen before all flips. Thus, after all swaps are done, the flips are fixed: any tile that has a different color than the target state must be flipped, and the other tiles must not.

Test Set 1

In Test Set 1, we can make the additional observation that no tile is to be swapped twice. Since we should not swap tiles when they are showing the same color, swapping a and b and then b and c is equivalent to flipping a and c , which costs the same. Because we swap each tile only once, we must only swap two tiles if the swap fixes both of their colors (the samples already hint at this fact). Since swaps fix colors for two tiles and flips for one tile, we want to do as many swaps as we can within this restrictions, and then flip the rest.

At this point, the problem becomes a classic one: some cells of the matrix need to be changed, and we want to match as many of those as possible with an adjacent cell that also needs to be changed, but to the other color. Since the graph of cells and adjacencies is [bipartite](#), this can be solved with a [maximum matching](#) algorithm on a bipartite graph. The solution is then the number of cells that need to change minus the size of such a maximum matching.

Test Set 2

As the additional sample shows, in Test Set 2 it is very possible that we need to swap the same tile multiple times. Luckily, we can generalize the solution explained for Test Set 1 for this scenario.

When a tile participates in multiple swaps, it effectively executes a swap with another tile that is not necessarily adjacent. While the other tiles are displaced by these operations, they are all of the same color, so that displacement has no effect. To put it in a different way, we can look at a swap as moving an M to an adjacent cell, and consecutive swaps are just consecutive moves.

Assume without loss of generality that the number of M s increases between the starting state and the target state (otherwise, use G s instead). Since we said we do all swaps first, that means that we move some of the M s that need to switch to G s to positions that have G s that need to be M s. We can do that by matching those two types of cell (regardless of adjacency). Any unmatched G into M position needs to be flipped (the number of these is fixed by the input).

For each matched pair, the cost of fixing both positions is either the minimum orthogonal distance between the two positions multiplied by S or $2F$, since we can also do two flips.

The reasoning above basically built a weighted bipartite matching between cells that start at M and need to be turned into G and cells that start at G and need to be turned into M . The total cost is the cost of the maximum matching of minimum weight plus F times the number of unmatched cells. We can apply any minimum cost maximum matching algorithm for bipartite graphs like the [Hungarian algorithm](#).